

Collections in VB

Why do we need collections?

A collection — sometimes called a container — is simply an object that groups multiple elements into a single unit. Collections are used to store, retrieve, manipulate, and communicate aggregate data.

What is difference between collection and array?

Arrays are fixed in size that is once we create an array we can not increased or decreased based on our requirement. Collection are growable in nature that is based on our requirement. We can increase or decrease of size.

Collections

Collections are objects in Visual Basic that are used to store a group of data values. Collections are similar to array. But there are some special features that differentiate the collections from arrays.

We have seen that array elements must be of the **same data type**. But the **Collection members can be of any data type and you don't need to set the size of the Collection object**. You can easily add items to the collection and it will grow accordingly.

Creating a Collection

To use a Collection in your code, you first need to declare and create it.

Example

```
Dim names As Collection      'Declaring the Collection  
Set names = New Collection   'Creating the Collection
```

Or, replace the above code with this one line code

```
'Declaration and creation
```

The Add method

You can add one item at a time using the Add method.

Example

```
Dim names As New Collection  
  
names.Add "john"  
  
names.Add "david"
```

You may use a string key associated with the item. The string key is used to refer to a Collection item.

```
Dim names As New Collection  
  
names.Add "John", "one"    ' "one" is the string key, used to refer to the item
```

The Item method

You can refer to a particular item of the collection using the Item method.
Here you may use either the index or key value. The index value starts from 1.

Syntax

```
Collection_Name.Item(Index)
```

or

```
Collection_Name.Item(Key)
```

Example

Retrieving a particular item using the index of the item

```
Private Sub cmdShow_Click()  
    Dim names As New Collection  
  
    names.Add "John", "one"  
    names.Add "David"  
  
    Print names.Item(2) '2 is the index of the item  
  
End Sub
```

Output

David

Example

Retrieving a particular item using the string key of the item

```
Private Sub cmdShow_Click()  
    Dim names As New Collection  
  
    names.Add "John", "one"  
    names.Add "David"  
  
    Print names.Item("one")  
  
End Sub
```

Output

John

The Item method is the default member of the Collection class, so you may omit it in your code.

Example

```
Private Sub cmdShow_Click()  
  
    Dim names As New Collection  
  
    names.Add "John", "one"  
    names.Add "David"  
  
    Print names("one")  
    Print names(2)  
  
End Sub
```

Output

```
John  
David
```

Before and After

You can choose to store the item values exactly where you want using the Before and After argument of the Add method.

Example

```
Private Sub cmdShow_Click()  
  
    Dim items As New Collection  
  
    items.Add "one"  
    items.Add "two"  
    items.Add "three", , 1  
  
    For i = 1 To 3
```

```
Print items.Item(i)

Next i

End Sub
```

Output

```
three

one

two
```

Remove

You can remove a particular item from the Collection using the Remove method.

Example

```
Private Sub Command1_Click()

    Dim country As New Collection

    country.Add "USA"

    country.Add "UK"

    country.Add "Japan", "j"

    country.Remove (1)

    country.Remove ("j")

End Sub
```

Count

Example

```
Private Sub cmdCount_Click()

    Dim country As New Collection

    country.Add "USA"
```

```
country.Add "India"

country.Add "Japan", "j"


Dim n As Integer

n = country.Count

Print n


End Sub
```

Output

3

Retrieving the last item

Example

```
Private Sub Command1_Click()

    Dim country As New Collection

    country.Add "Germany"

    country.Add "India"

    country.Add "China", "j"


    Print country.Item(country.Count)

End Sub
```

Output

China

Delete all items

Use a Do While loop to delete all item from the Collection object.

Example

```
Private Sub cmdDeleteAll_Click()

    Dim country As New Collection

    country.Add "Bangladesh"

    country.Add "Australia"

    country.Add "Russia", "j"

    Do While (country.Count > 0)

        country.Remove 1

    Loop

    Print country.Count

End Sub
```

Output

0

Another way to delete all items is to destroy the Collection object. The following code destroys the Collection object and thus deletes all the items.

```
Set items = Nothing
```

Or

```
Set items = New Collection
```

What is the difference between array and collection in vb6?

Collections are similar to array. But there are some special features that differentiate the collections from arrays. We have seen that array elements must be of the same data type. But the Collection members can be of any data type and you don't need to set the size of the Collection object.

Which is faster array or collection?

An array due to fast execution consumes more memory and has better performance. Collections, on the other hand, consume less memory but also have low performance as compared to Arrays. **Arrays can hold the only the same type of data in its collection i.e only homogeneous data types elements are allowed in case of arrays.**

What is procedure in vb6?

A procedure is a block of Visual Basic statements enclosed by a declaration statement (Function , Sub , Operator , Get , Set) and a matching End declaration. All executable statements in Visual Basic must be within some procedure.